Unit IV

4. Frequency Domain Processing

4.1 Introduction to Discrete Fourier Transformation (DFT)
4.2 Computing and visualizing 1D-DFT
4.3 Computing and visualizing 2D-DFT
4.4 Filtering in frequency domain
4.5 Color Image Representation
4.6 Converting to other color spaces
4.7 The Basics of color image processing

4.1 Introduction to Discrete Fourier Transformation (DFT):

The **Discrete Fourier Transformation (DFT)** is a fundamental mathematical technique used in digital signal and image processing to convert a signal from the **spatial (time or image) domain** into the **frequency domain**. It analyzes how different frequency components contribute to the overall signal.

Definition:

DFT transforms a sequence of discrete values into an equal-length sequence representing its frequency components. For an input sequence $f(n)$, the DFT is given by:

$$F(k) = \sum_{n=0}^{N-1} f(n) e^{-j2\pi kn/N}$$

Here,

$f(n)$: Input signal
$F(k)$: Frequency-domain output
N: Number of samples

Key Concepts:

1. **Frequency Representation:**
   DFT shows how much of each frequency is present in the signal, separating it into low, medium, and high frequencies.

2. **Reversibility:**
   The original signal can be reconstructed using the inverse DFT (IDFT).

3. **Magnitude and Phase:**
   Each frequency component has a **magnitude** (strength) and **phase** (position shift).

4. **Complex Output:**
   The DFT produces complex numbers due to sinusoidal decomposition using sine and cosine waves.

Applications:

Image filtering and enhancement
Noise reduction
Feature extraction
Compression
Pattern and frequency analysis

Conclusion:

DFT is a powerful tool that converts signals into their frequency components, enabling efficient analysis, filtering, and processing of digital images and signals.

4.2 Computing and visualizing 1D-DFT:

1. Computing 1D-DFT:

The **1-Dimensional Discrete Fourier Transform (1D-DFT)** is used to convert a discrete signal from the **time (or spatial) domain** into the **frequency domain**. Computing the DFT involves applying a mathematical formula that breaks the signal into sinusoidal components of different frequencies.

DFT Formula:

For a discrete sequence $f(n)$ of length N, the DFT is computed as:

$$F(k) = \sum_{n=0}^{N-1} f(n)e^{-j2\pi kn/N}, \quad k = 0, 1, 2, \ \dots, \ N-1$$

Steps in Computing 1D-DFT:

1. **Take the input signal** (e.g., a list of intensity or amplitude values).
2. **Multiply each sample** with complex exponential basis functions.
3. **Sum all the products** to obtain a frequency component $F(k)$.
4. **Repeat for all values of k** to compute the full spectrum.
5. The output is a set of **complex values** representing magnitude and phase.

Characteristics:

Produces both **real and imaginary parts**.
Shows how strongly each frequency exists in the signal.
Time-domain convolutions become multiplications in frequency domain.

MATLAB Example:

```
x = [1 2 3 4];
X = fft(x);
```

Conclusion:

Computing 1D-DFT is essential for analyzing the frequency content of signals, enabling filtering, compression, and spectral analysis.

2. Visualizing 1D-DFT:

Once the 1D-DFT is computed, it is important to **visualize** the frequency components for better understanding and analysis. Visualization shows the **magnitude** and **phase** of different frequency elements present in a signal.

Key Concepts in Visualization:

1. **Magnitude Spectrum:**

   Displays the strength (amplitude) of each frequency component.
   Magnitude is calculated as:

$$|F(k)| = \sqrt{(real\ (F(k)))^2 + (imag\ (F(k)))^2}$$

2. **Phase Spectrum:**

   Shows the phase shift of each frequency component.
   Phase is calculated as:

$$< F(k) = tan^{-1}\left(\frac{imag\ (F(k))}{real\ (F(k))}\right)$$

3. **Frequency Axis:**

   The x-axis represents discrete frequency values from 0 to $N-1$.

MATLAB Example for Visualization:

```
x = [1 2 3 4];
X = fft(x);
```

```
subplot(2,1,1);
plot(abs(X));   % Magnitude spectrum
title('Magnitude Spectrum');

subplot(2,1,2);
plot(angle(X)); % Phase spectrum
title('Phase Spectrum');
```

Importance of Visualization:

Helps identify dominant frequencies
Useful for detecting noise components
Allows comparison of signals
Essential in communications, audio processing, and image analysis

Conclusion:

Visualizing 1D-DFT provides clear insight into both the magnitude and phase characteristics of a signal, making it a crucial step in frequency-domain analysis.

4.3 Computing and visualizing 2D-DFT:

The **2-Dimensional Discrete Fourier Transform (2D-DFT)** is used in digital image processing to convert an image from the **spatial domain** (pixels) to the **frequency domain**. It helps analyze how different frequency components contribute to the image structure.

1. Computing 2D-DFT:

For an image $f(x, y)$ of size $M \times N$, the 2D-DFT is computed using:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi \left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

Steps:

1. Take the input image as a 2D array of pixel values.
2. Multiply each pixel by complex exponential basis functions.
3. Sum the products over all rows and columns to obtain frequency components.
4. Output is a complex-valued matrix representing frequency content.
5. In MATLAB, 2D-DFT is computed using `fft2()`.

MATLAB Example:

```
img = imread('image.jpg');
F = fft2(img);          % compute 2D-DFT
F_shift = fftshift(F);  % shift low frequencies to center
```

2. Visualizing 2D-DFT:

Visualization helps interpret frequency information such as edges, textures, and smooth areas.

Key Concepts:

1. **Magnitude Spectrum:**

   Shows the strength of each frequency component. High values correspond to strong edges or details.

$$|F(u, v)| = \sqrt{(Real)^2 + (Imag)^2}$$

2. **Log Transform:**

   Used because magnitude values vary widely.

$$\log(1 + |F(u, v)|)$$

3. **Phase Spectrum:**
Represents spatial alignment; essential for reconstructing the image.

MATLAB Example for Visualization:

```
figure;
imshow(log(1 + abs(F_shift)), []);
title('Magnitude Spectrum');

figure;
imshow(angle(F_shift), []);
title('Phase Spectrum');
```

Interpretation:
Bright areas in magnitude spectrum = strong frequency components.
Center represents low frequencies (smooth areas).
Corners represent high frequencies (edges, sharp transitions).

Conclusion:
Computing and visualizing 2D-DFT provides a powerful way to analyze image frequency content, identify edges and textures, and perform operations such as filtering, enhancement, and compression.

4.4 Filtering in frequency domain:

Filtering in the frequency domain is a technique used in digital image processing to **modify specific frequency components** of an image to achieve effects such as smoothing, sharpening, and noise reduction.

Concept:
An image is first transformed from the spatial domain to the **frequency domain** using the **Discrete Fourier Transform (DFT)**. In this domain, image details are represented as frequency components:
Low frequencies: smooth areas
High frequencies: edges, noise, fine details

Filtering is done by **multiplying** the image's Fourier transform with a **frequency-domain filter mask**.

Steps in Frequency-Domain Filtering:

1. **Take the image** $f(x, y)$.

2. **Compute its 2D-DFT:**
$$F(u, v) = DFT\{f(x, y)\}$$

3. **Design a filter** $H(u, v)$ (low-pass, high-pass, band-pass, etc.).

4. **Multiply:**
$$G(u, v) = H(u, v) \cdot F(u, v)$$

5. **Apply inverse DFT** to obtain the filtered image:
$$g(x, y) = IDFT\{G(u, v)\}$$

Types of Frequency-Domain Filters:

1. **Low-Pass Filters (LPF):**
   Allow low frequencies; remove noise and smooth the image.
   Examples: Ideal LPF, Gaussian LPF.

2. **High-Pass Filters (HPF):**
   Allow high frequencies; enhance edges and fine details.
   Examples: Ideal HPF, Butterworth HPF.

3. **Band-Pass and Band-Stop Filters:**
   Allow or block specific frequency ranges.

Advantages:

More precise control over frequency components
Effective for smoothing and sharpening
Useful in image enhancement and restoration

Conclusion:

Filtering in the frequency domain is a powerful method that uses the Fourier transform to selectively modify frequency components, enabling tasks like noise removal, edge enhancement, and overall image quality improvement.

4.5 Color Image Representation:

Color image representation refers to the method by which color information is stored, processed, and displayed in digital images. Unlike grayscale images that contain only intensity values, color images consist of multiple components representing different color channels.

1. RGB Color Model:

The most widely used representation is the **RGB (Red, Green, Blue)** model.

A color image is stored as **three separate 2D matrices** (one for each color channel).
Each pixel is represented by a combination of **R, G, and B intensities**, typically in the range 0 - 255.
These three channels combine to produce a full-color image.

2. True Color Image:

A standard color image uses **24-bit representation**, where:

8 bits → Red
8 bits → Green
8 bits → Blue
This allows **16.7 million** possible colors (2^{24}).

3. Other Color Models:

Besides RGB, several models are used for specific applications:

HSV (Hue, Saturation, Value): represents color in terms of shade, colorfulness, and brightness.
CMY/CMYK: used in printing.
YCbCr: used in video compression and broadcasting.
Different models separate brightness information from color information, useful for enhancement and compression.

4. Pixel Structure:

Each pixel in a color image is represented as a **vector**:

$Pixel = [R, G, B]$

Thus, a color image can be seen as a **3D array** (height × width × 3).

5. Applications:

Computer vision
Image editing
Medical imaging
Remote sensing
Graphics and multimedia

Conclusion:

Color image representation uses multiple color channels (typically RGB) to store and display color information. This multidimensional data structure allows accurate reproduction, processing, and analysis of colored images.

4.6 Converting to other color spaces:

Color images are most commonly represented in the **RGB color model**, but many image processing applications require converting images into other color spaces such as **HSV, YCbCr, CMY, or Lab**. This conversion allows better manipulation of brightness, contrast, and color information.

1. Need for Color Space Conversion:

Different color spaces separate color and intensity information differently. Converting RGB images to other color models helps in:

Image segmentation
Object recognition
Compression
Color enhancement
Video processing

2. Common Color Spaces and Their Use:

1. **HSV (Hue, Saturation, Value):**
   Separates color information (Hue) from brightness (Value).
   Useful in color-based segmentation, tracking, and filtering.

2. **YCbCr:**
   Separates luminance (Y) from chrominance (Cb, Cr).
   Used in image and video compression (JPEG, MPEG).

3. **CMY/CMYK:**
   Used in color printing.
   Derived from RGB by subtracting each channel from 1.

4. **Lab Color Space:**
   Designed to be perceptually uniform.
   Useful in color correction and enhancement.

3. Conversion Example (RGB to HSV):

Hue (H): determines the type of color
Saturation (S): intensity of color
Value (V): brightness

MATLAB uses:

```
hsvImage = rgb2hsv(rgbImage);
```

4. Conversion Example (RGB to YCbCr):

```
ycbcrImage = rgb2ycbcr(rgbImage);
```

5. Advantages of Converting Color Spaces:

Simplifies tasks like detection and segmentation
Makes brightness-independent processing easy
Improves compression efficiency
Enhances color analysis and manipulation

Conclusion:

Converting RGB images to other color spaces is essential in image processing because different models separate color and intensity information in useful ways. These conversions enable more effective analysis, processing, and enhancement of color images.

4.7 The Basics of color image processing:

Color image processing involves techniques used to represent, manipulate, and analyze color images. Unlike grayscale images, which contain only intensity information, color images include multiple channels representing different color components.

1. Color Image Representation:

A color image is typically represented using **three separate color channels**.
Example:

RGB Model: Red, Green, and Blue components combine to form a full-color image.
A color pixel is represented as a vector:

$$[R, G, B]$$

2. Color Models (Color Spaces):

Different applications use different color models:

RGB: Used for display devices like monitors and cameras.
HSV (Hue, Saturation, Value): Separates color from brightness; useful for segmentation.
YCbCr: Used in video and image compression.
CMYK: Used in printing.

3. Color Transformations:

Changing between color spaces (e.g., RGB → HSV) helps in tasks like detection, enhancement, and filtering. MATLAB functions like `rgb2gray`, `rgb2hsv`, and `rgb2ycbcr` support such transformations.

4. Color Image Enhancement:

Operations such as contrast adjustment, histogram equalization of individual channels, and smoothing/sharpening are applied to improve visual quality.

5. Color Image Segmentation & Analysis:

Color information helps identify objects based on hue or saturation—for example, detecting skin color, traffic signs, or vegetation in images.

Conclusion:

Color image processing forms the foundation of digital image analysis. It uses color models, transformations, and enhancement techniques to represent and manipulate color information for applications in vision, graphics, multimedia, and pattern recognition.